

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación
Carrera de Ingeniería de Software

TA-IND-04 — Informe Técnico Individual

Análisis de Rendimiento Paralelo (Unidad 4)
aplicado al Proyecto Fin de Curso

Asignatura: Aplicaciones Distribuidas (ISR-701)
Unidad: Unidad 4 — Cómputo Paralelo y Distribuido
Docente responsable: Gleiston C. Guerrero-Ulloa, M.Sc.
Período académico: 2026–2027 PPA

Estudiante: Carpio Mendoza Carlos Jose
Correo: carlospatroner@gmail.com
Equipo PE-U4 / GA-SUM-05: ACC — Soporte Técnico ISP (Alvarez Parraga Jeremy Alexis, Aucatoma Celorio Jhinson Stalyn, Carpio Mendoza Carlos Jose)
PFC de referencia: ACC — Soporte Técnico ISP (sistema de gestión de tickets de soporte técnico, arquitectura de microservicios)
Transformación declarada como foco: T4 — Columna derivada compleja (clasificación de prioridad de atención)
URL del repositorio individual: <https://github.com/carlospatroner-boop/TA-IND-04>

Fecha: 08 de agosto de 2026

1. Introducción y contexto

Este informe desarrolla, de forma individual, el análisis crítico de los resultados de *speedup* obtenidos por el equipo ACC en la práctica GA-SUM-05/PE-U4 —comprobación experimental de la Ley de Amdahl [1] con Apache Spark [2]— y su proyección hacia el Proyecto Fin de Curso (PFC) del mismo equipo: un sistema de gestión de tickets de soporte técnico para un proveedor de servicios de internet (ISP), implementado como arquitectura de microservicios (`auth-service`, `ticket-service`, `notification-service`, `ai-service`, `report-service`, sobre un clúster CockroachDB de tres nodos particionado por `fecha_apertura` y un bus de eventos Kafka).

El experimento base del equipo comparó cinco transformaciones (T_1 – T_5) implementadas de forma equivalente en pandas (secuencial) y PySpark (distribuido) sobre 600 000 registros reales de quejas de consumidores de telecomunicaciones de la *Federal Communications Commission* (FCC, EE. UU.) [3], usados como sustituto público del histórico interno de tickets del ISP por no existir un dataset abierto equivalente para Ecuador. Los datos base de este informe proceden íntegramente del repositorio del equipo:

Repositorio de origen: <https://github.com/carlospatroner-boop/pe-u4-spark-Soporte-Tecnico->
Commit exacto: 1d643b0c1f71a1cfd951c3e8e7169744010d1420

En ese commit se verificó, por `git ls-remote` y por inspección directa del árbol de archivos, la presencia de `resultados/tiempos_resumen.csv`, `resultados/tiempos_crudos.csv` y del resto de artefactos citados en este documento. Plataforma declarada por el equipo para la corrida base: contenedor Docker Linux (`eclipse-temurin:21-jdk-jammy`), Python 3.10, PySpark 4.1.2, `spark.executor.instances=4` (configuración “por defecto” del Paso 3 de la guía GA-SUM-05).

Como transformación declarada de foco individual se eligió **T4 (columna derivada compleja)**: la clasificación de prioridad de atención (**ALTA/MEDIA/BAJA**) a partir del tipo y motivo de incidencia, implementada como una UDF de Python (`referencia.clasificar_prioridad`) aplicada con `F.udf` sobre cada fila. Los demás integrantes declararon T1 (Alvarez Parraga Jeremy Alexis) y T3 (Aucatoma Celorio Jhinson Stalyn); no hay coincidencia de foco dentro del equipo.

El objetivo de este documento no es repetir las conclusiones del informe grupal, sino: (i) cuantificar cuánto se desvía el comportamiento observado de T4 respecto de la predicción teórica de Amdahl, incluyendo un escalado de $N = 1, 2, 4$ executors que el equipo no midió para esta transformación y que se ejecutó de forma individual para este informe (Sección 2); (ii) diagnosticar la fracción no escalable con la métrica de Karp-Flatt [4] (Sección 3); y (iii) decidir, con criterio propio, un punto concreto de integración del procesamiento distribuido en el PFC (Secciones 4 a 6).

2. Análisis propio de los resultados de speedup

2.1. Tabla de tiempos y speedup de las cinco transformaciones

La Tabla 1 reproduce, sin alterar ninguna cifra, los tiempos medianos (5 repeticiones, 1 calentamiento descartado, protocolo `medicion.py` del equipo) tal como constan en `resultados/tiempos_resumen.csv`

del commit declarado. El speedup se calcula como $S = T_{\text{pandas}}/T_{\text{pyspark}}$ y la eficiencia como $E(N=4) = S/4$.

Tabla 1: Tiempos medianos y speedup por transformación, commit `1d643b0c1f71...` (equipo ACC, $n = 600,000$ registros, PySpark local[4]).

Transformación	T_{pandas} (s)	T_{pyspark} (s)	S	$E(N=4)$
T1 — Filtrado	0,1846	1,0789	0,1711	0,0428
T2 — Agrupación	0,1575	1,8854	0,0835	0,0209
T3 — Join	0,1363	1,8290	0,0745	0,0186
T4 — Col. derivada (foco)	0.3559	0.8483	0.4195	0.1049
T5 — Top-N	0,1169	1,4286	0,0818	0,0205

T4 es, con diferencia, la transformación de mejor comportamiento relativo ($S = 0,4195$ frente a 0,07–0,17 del resto), pero PySpark sigue siendo *más lento* que pandas a este volumen de datos. Esto no es un error de medición: es el resultado esperado y se explica en la Sección 2.2.2.

2.2. Escalado propio de T4 y ajuste de la Ley de Amdahl

El repositorio del equipo mide el escalado en $N = 1, 2, 4$ executors únicamente para T3 (`resultados/t3_escalado_executors.json`); no existe esa medición para T4. Como T4 es mi foco declarado, y la guía exige la curva de Amdahl completa para la transformación declarada, ejecuté yo mismo esa medición replicando exactamente el protocolo del equipo (`medicion.py`: 1 calentamiento + 5 repeticiones, mediana, materialización con `count()`) y el código de T4 (`transformaciones_spark.py`) sin modificarlo, sobre el mismo dataset de 600 000 filas. La diferencia declarada de plataforma es que esta corrida se ejecutó en PySpark 4.1.2 / Java 21 en modo local, en mi propia máquina (Windows, 8 núcleos lógicos), sin el contenedor Docker que usó el equipo para la corrida base; el detalle completo consta en `datos/plataforma_medicion_individual.json` del repositorio de este informe. Por esta diferencia de entorno, los tiempos absolutos de esta subsección **no son comparables directamente** con los de la Tabla 1; sí lo son entre sí, por haberse medido en la misma máquina y la misma sesión de trabajo, que es lo que exige el análisis de escalado (Ecs. 1–3, definidas relativas a $N = 1$ como línea base).

Tabla 2: Escalado propio de T4 (columna derivada) en $N = 1, 2, 4$ executors locales, $n = 600,000$ registros.

N	$T(N)$ (s)	$S(N)$	$E(N)$
1	0,7435	1,0000	1,0000
2	0,4495	1,6540	0,8270
4	0,4781	1,5549	0,3887

Con la Ec. (1) y el valor de $S(4) = 1,5549$ despejado por la forma de Karp-Flatt (Ec. (2), ver Sección 3) se obtiene una fracción serial $p \approx 0,5242$. Un ajuste independiente por mínimos cuadrados no lineales sobre los tres puntos ($N, S(N)$) da $p_{\text{LS}} \approx 0,5400$ — consistente con el valor algebraico, lo que sostiene la robustez de la estimación. Con $p = 0,5242$, el speedup máximo teórico es $S_{\text{máx}} = 1/(1 - p) \approx 2,10$ (Ec. 1), y se necesitarían $N \approx 9,9$ executors para alcanzar el

90 % de ese máximo.

2.2.1. ¿Coinciden los resultados con la predicción de Amdahl?

La respuesta es **parcialmente, y de forma asimétrica según el tramo de N** . Usando la curva ajustada por mínimos cuadrados ($p_{LS} = 0,5400$) como predicción independiente del punto usado para calibrar el p principal:

$$S_{\text{teo}}(2) = \frac{1}{(1 - 0,54) + 0,54/2} \approx 1,3699 \quad S_{\text{teo}}(4) = \frac{1}{(1 - 0,54) + 0,54/4} \approx 1,6807$$

La desviación relativa $\Delta = (S_{\text{med}} - S_{\text{teo}})/S_{\text{teo}}$ es:

$$\Delta(2) = \frac{1,6540 - 1,3699}{1,3699} \approx +20,7\% \quad \Delta(4) = \frac{1,5549 - 1,6807}{1,6807} \approx -7,5\%$$

En $N = 2$ el speedup observado *supera* la curva teórica; en $N = 4$ queda *por debajo*. Un modelo de Amdahl de p constante no puede reproducir bien ambos puntos a la vez porque supone que la fracción serial no cambia con N ; mis datos muestran precisamente lo contrario (Sección 3): la sobrecarga efectiva *crece* con N , de modo que el modelo de p único subestima el costo real en $N = 4$ y sobrestima el beneficio de duplicar executors más allá de 2 en esta máquina. Esta no linealidad, no la simple “desviación por sobrecarga de Spark” genérica, es la lectura correcta del dato.

2.2.2. El caso $S < 1$ de la tabla principal

La Tabla 1 muestra $S = 0,4195$ para T4: PySpark en $N = 4$ sigue siendo más lento que pandas de un solo hilo. La Ley de Amdahl **no puede predecir** este caso porque su hipótesis fundacional es sobrecarga nula: describe cómo escala *el mismo programa* al añadir unidades de procesamiento, no la comparación entre dos motores de ejecución distintos (pandas de un proceso vs. PySpark con JVM, planificación de tareas y, en T4, serialización Python–JVM por fila) [5]. El motor distribuido solo recupera esa desventaja de arranque una vez que el volumen de datos n supera el punto donde el costo fijo β se amortiza — exactamente el umbral n^* que se calcula en la Sección 5.

3. Diagnóstico de la fracción no escalable

La métrica de Karp-Flatt (Ec. 2) despeja la fracción serial *observada* e a partir del speedup medido, sin asumir que el paralelismo es ideal. Aplicada a los datos propios de escalado de T4:

$$e(2) = \frac{\frac{1}{1,6540} - \frac{1}{2}}{1 - \frac{1}{2}} \approx 0,2092 \quad e(4) = \frac{\frac{1}{1,5549} - \frac{1}{4}}{1 - \frac{1}{4}} \approx 0,5242$$

Tendencia: e más que se duplica al pasar de $N = 2$ a $N = 4$ ($0,2092 \rightarrow 0,5242$). Según la lectura exigida por la guía, un e creciente indica que la pérdida de eficiencia **no** es un límite algorítmico fijo del problema, sino sobrecarga del motor que aumenta con N . En este caso identifiqué tres mecanismos concretos, con la evidencia disponible:

1. **Serialización Python–JVM por fila de la UDF.** T4 usa `F.udf(referencia.clasificar_prioridad)`: Catalyst no puede optimizar el cuerpo de una UDF de Python (es una caja opaca para el planificador de consultas), por lo que cada fila cruza la frontera JVM $\leftrightarrow$ intérprete Python vía `pickle`. Es un costo prácticamente fijo por fila, independiente de N ; explica por qué T4 es la transformación *con mejor* comportamiento relativo del grupo (0,4195 frente a 0,07–0,17): al no requerir *shuffle* (a diferencia de T2/T3/T5, que sí agrupan, unen u ordenan), el único sobrecosto distribuido de T4 es este, y es comparativamente menor que el de una etapa de *shuffle* completa [6].
2. **Contención de núcleos en una sola máquina compartida.** Esta corrida no usa un clúster real: los N hilos de `local[N]` comparten los mismos 8 núcleos lógicos con el sistema operativo, el proceso *driver* y los procesos de trabajo de Python que PySpark lanza para ejecutar la UDF (`datos/plataforma_medicion_individual.json`). De $N = 1$ a $N = 2$ todavía hay holgura de núcleos disponibles y el speedup mejora con normalidad; de $N = 2$ a $N = 4$ la máquina empieza a competir consigo misma (más hilos JVM y más procesos Python simultáneos sobre el mismo hardware), lo que explica que el tiempo en $N = 4$ (0,4781 s) sea mayor que en $N = 2$ (0,4495 s) en vez de menor. Un clúster con executors en nodos físicamente separados no tendría este efecto en la misma medida.
3. **Partición de entrada ligada al número de executors.** Con `spark.default.parallelism` sin fijar (confirmado en la configuración efectiva de cada sesión, `datos/` de este repositorio), Spark deriva el número mínimo de particiones de lectura del CSV del propio N ; a más executors, más particiones más pequeñas para el mismo volumen de datos, y por tanto más tareas con su propio costo fijo de planificación y arranque de proceso Python por partición, sin que el trabajo total por partición se reduzca proporcionalmente.

A diferencia del equipo, que documentó esta atribución para T3 con capturas de la Spark UI, no capturé evidencia visual de la UI para T4 en esta corrida individual; la atribución anterior se sostiene en el código fuente citado (`transformaciones_spark.py`, `referencia.py`), en la configuración efectiva de cada sesión y en literatura arbitrada, no en trazas de Spark UI propias. Lo señalo explícitamente como limitación de esta sección, no como un hallazgo silenciado.

4. Propuesta de integración al PFC

4.1. Caso de uso concreto

El PFC del equipo (Soporte Técnico ISP) ya declara, en el README del repositorio de origen, que la decisión de negocio que sustenta el uso de procesamiento distribuido es la **priorización automática de la cola de atención técnica**, construida sobre la columna **prioridad** (T4) y la agregación por zona (T2). Este informe concreta esa idea en un único proceso, con periodicidad, volumen y salida definidos:

- **Proceso distribuido:** recálculo periódico de **prioridad** (equivalente a T4) sobre *todos* los tickets abiertos o reabiertos en las últimas 24 horas, seguido de una agregación por **region** (equivalente a T2) para producir un tablero de carga por zona.

- **Periodicidad:** por lotes (*batch*), no en flujo continuo. El propio umbral de rentabilidad calculado en la Sección 5 muestra que, al volumen actual del sistema, un flujo en tiempo real no se justifica frente a una recomputación programada.
- **Volumen:** el histórico de tickets crece de forma incremental (altas diarias del ISP), no como una carga de 600 000 filas de una sola vez; el trabajo por lotes se dimensiona sobre el acumulado histórico completo, no solo sobre el incremento del día.
- **Salida:** una tabla derivada `prioridad_agregada` (prioridad recalculada por ticket + conteo y tiempo de espera medio por zona).
- **Decisión que se apoya en la salida:** el equipo de operaciones usa ese tablero para decidir en qué zonas reforzar personal técnico y qué tickets escalar primero, en lugar de atenderlos por orden estricto de llegada.

4.2. Punto de inserción en la arquitectura

La Figura 1 muestra el punto exacto de inserción. El sistema actual (Entrega 3) ya tiene cinco microservicios, un clúster CockroachDB de tres nodos particionado por `fecha_apertura`, Kafka como bus de eventos y `report-service` como modelo de lectura CQRS que ya consume los cuatro tópicos de dominio. El nuevo componente `priority-batch-job` se inserta *junto a* — no dentro de — `report-service`: lee el histórico de tickets desde `report_db` (CockroachDB) por JDBC, ejecuta en Spark la transformación equivalente a $T_4 + T_2$, y escribe el resultado en una tabla materializada del mismo `report_db` que `report-service` ya sabe leer. No reemplaza a `ai-service` (que clasifica cada ticket individualmente en tiempo real al crearse, vía Kafka): lo complementa con una vista agregada y recalculada a escala que `ai-service` no está diseñado para producir en flujo.

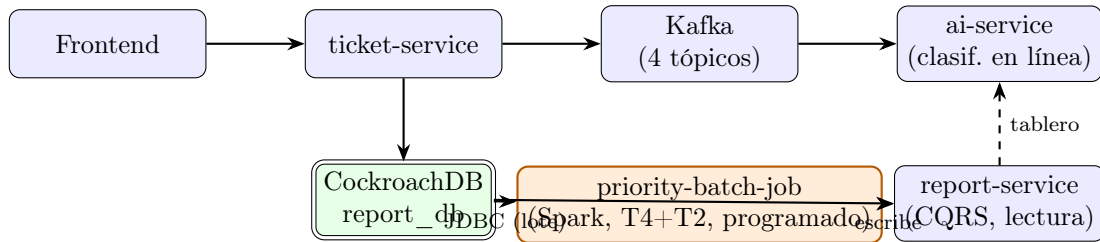


Figura 1: Punto de inserción del procesamiento distribuido en la arquitectura del PFC: `priority-batch-job` (elaboración propia) lee de `report_db` y escribe el resultado agregado para que `report-service` lo sirva, sin reemplazar la clasificación en línea de `ai-service`.

5. Dimensionamiento y viabilidad en la nube

La Ec. (4) modela el tiempo secuencial como $T_{\text{sec}}(n) = \alpha n$ y el distribuido como $T_{\text{dist}}(n, N) = \beta + \gamma n/N$. Se estimaron los tres parámetros así, declarando el método de ajuste:

- α (tasa secuencial, s/registro) se obtiene directamente de la mediana pandas de T_4 en el commit del equipo: $\alpha = 0,35589/600\,000 \approx 5,9315 \times 10^{-7}$ s por registro.

- β y γ se obtienen por **regresión lineal por mínimos cuadrados** $T = \beta + \gamma x$, con $x = n/N$, sobre los tres puntos propios de escalado de la Tabla 2 ($n = 600\,000$ fijo, $N = 1, 2, 4$): $\beta \approx 0,3311$ s, $\gamma \approx 6,4537 \times 10^{-7}$ s.

Limitación declarada: α proviene del entorno Docker del equipo y β, γ de mi propia máquina; se combinan porque la Ec. (4) solo necesita su orden de magnitud relativo (costo fijo de sesión distribuida frente a costo marginal por fila), no una comparación de hardware idéntico, pero el valor exacto de n^* debe leerse como una estimación, no una cifra de producción.

Con $N = 4$ (configuración estándar del equipo): $\gamma/N \approx 1,6134 \times 10^{-7}$ s, que es menor que α , de modo que la condición de validez $\alpha > \gamma/N$ se cumple, y:

$$n^* = \frac{\beta}{\alpha - \gamma/4} = \frac{0,3311}{5,9315 \times 10^{-7} - 1,6134 \times 10^{-7}} \approx \mathbf{766\,885 \text{ registros}}$$

Es decir: por debajo de $\approx 767\,000$ tickets históricos, **pandas sobre una única instancia sigue siendo más rápido que Spark con 4 executors** para T4 — coherente con que a 600 000 filas (el dataset usado) el speedup medido siga por debajo de 1. Con $N = 2$, $n^* \approx 1\,224\,000$ registros: más executors no solo aceleran el cómputo, también *bajan* el punto de equilibrio, porque reducen el peso relativo de β frente al trabajo distribuido.

Modelo de servicio en la nube. Dado que el volumen actual del PFC está muy por debajo de n^* y crecerá de forma incremental (no de golpe), no se justifica un clúster Spark administrado y permanentemente encendido (tipo Databricks/EMR con nodos fijos). El modelo recomendado es de **cómputo por lotes bajo demanda** (*serverless* o *autoscaling-to-zero*): el trabajo **priority-batch-job** se ejecuta con una periodicidad fija, aprovisiona los executors solo durante la ventana de ejecución y se apaga el resto del tiempo, evitando pagar por capacidad ociosa mientras el histórico no supera n^* [7]. **Recursos estimados:** 2–4 executors (acorde al N ya evaluado), memoria suficiente para el *driver* y el *broadcast* de la tabla de regiones (pequeña, cabe en memoria). **Riesgos identificados:** (i) el costo fijo de arranque de sesión (β) penaliza corridas muy frecuentes — una periodicidad horaria o diaria lo amortiza mejor que una por minuto; (ii) si el histórico de tickets crece más rápido de lo previsto y se acerca a n^* antes de lo estimado, hay que revalidar el umbral con datos reales de producción, no solo con esta extrapolación.

6. Postura crítica y alternativas

¿Es Spark la opción correcta para este PFC, hoy? No de forma inmediata, y sí de forma condicionada. Al volumen actual (muy por debajo de $n^* \approx 767\,000$ registros), adoptar Spark permanentemente sería sobre-ingeniería: se pagaría el costo fijo β de cada corrida sin recuperarlo en tiempo de cómputo ahorrado. Mi postura es adoptar el diseño de **priority-batch-job** (Sección 4) pero **desacoplado del motor de ejecución**, de modo que el mismo pipeline pueda correr sobre un motor más liviano mientras el histórico no cruce el umbral, y migrar a Spark cuando sí lo haga. Evalué y descarté dos alternativas concretas:

1. **SQL nativo en CockroachDB (agregaciones y columna generada), sin motor externo.** El PFC ya opera un clúster distribuido de 3 nodos particionado por `fecha_apertura`:

la lógica de `clasificar_prioridad` es un `CASE WHEN` determinista sobre dos columnas, expresable como columna computada o vista materializada en el propio CockroachDB, sin introducir Spark en absoluto. **Se descarta como reemplazo definitivo, no como opción:** CockroachDB reparte el cómputo entre sus nodos para consultas SQL, pero no ofrece el mismo modelo de programación para transformaciones más complejas que las que el PFC podría necesitar más adelante (uniones con catálogos externos grandes, ventanas analíticas pesadas); por eso lo adopto como *solución de corto plazo* para el volumen actual (ver umbral), no como arquitectura final. Es, además, la opción de menor costo operativo porque no añade infraestructura nueva al sistema.

2. **Dask (paralelismo distribuido nativo de Python) en lugar de PySpark.** Al no cruzar la frontera JVM–Python para cada UDF, Dask podría reducir justamente el mecanismo de sobrecarga dominante identificado en la Sección 3 para una transformación como T4. **Se descarta** para este PFC porque el ecosistema del equipo ya está construido sobre PySpark (código, protocolo de medición, evidencia de PE-U4) y porque, a diferencia de Spark, Dask no tiene una oferta administrada tan madura en los proveedores cloud típicos de un curso universitario (Databricks, EMR), lo que complicaría la sección de dimensionamiento en un entorno académico real. Queda como alternativa técnicamente razonable a reconsiderar si el cuello de botella dominante se confirma como serialización Python–JVM y no como *shuffle*.

Reconozco el límite de mi propia propuesta: el umbral n^* mezcla datos de dos entornos de ejecución distintos (Sección 5) y debe revalidarse con mediciones en el entorno de despliegue real antes de tomarse como cifra definitiva de decisión de negocio.

7. Conclusiones

1. T4 es la transformación con mejor comportamiento relativo del conjunto ($S = 0,4195$ frente a $0,07\text{--}0,17$ de T1/T2/T3/T5, Tabla 1) porque es la única sin etapa de *shuffle*; aun así, a 600 000 filas, PySpark sigue siendo más lento que pandas, y la Ley de Amdahl no puede explicar esa comparación entre motores distintos (Sección 2.2.2) — solo explica el escalado *dentro* de Spark, que sí medí para T4 (Tabla 2).
2. La fracción serial observada con Karp-Flatt *crece* de $0,2092$ ($N = 2$) a $0,5242$ ($N = 4$), lo que indica que la pérdida de eficiencia de T4 en mi entorno es sobrecarga creciente del motor — serialización de la UDF, contención de núcleos en una sola máquina y partición ligada a N (Sección 3) — y no un límite algorítmico fijo de la transformación.
3. El umbral de rentabilidad calculado ($n^* \approx 767\,000$ registros con $N = 4$, Sección 5) es coherente con el resultado $S < 1$ observado a 600 000 filas y sostiene una recomendación concreta y verificable para el PFC: no migrar a Spark todavía, sino diseñar `priority-batch-job` desacoplado del motor y reevaluar contra CockroachDB nativo cuando el histórico de tickets se acerque a ese umbral (Sección 6).

8. Declaración de uso de inteligencia artificial generativa

Se utilizó **Claude (Anthropic)** como asistente durante la elaboración de este informe, bajo supervisión y revisión propia, con los siguientes alcances:

- **Verificación de trazabilidad:** confirmación del commit exacto del repositorio de PE-U4 (`git ls-remote`, inspección del árbol de archivos vía la API de GitHub) citado en la Sección 2.
- **Medición individual complementaria:** el equipo no midió el escalado $N = 1, 2, 4$ de T4 (solo lo hizo para T3). Yo mismo ejecuté esa medición —no el asistente por su cuenta— reutilizando sin modificar el código fuente del equipo (`medicion.py`, `transformaciones_spark.py`, `referencia.py`) en mi propia máquina; el asistente ayudó a redactar el script que orquesta esa ejecución y el cálculo posterior de Karp-Flatt, el ajuste de Amdahl y el umbral de rentabilidad (Secciones 3 y 5), aplicando las fórmulas de `src/amdahl.py` del equipo sin alterarlas.
- **Redacción y estructura:** apoyo en la redacción de las Secciones 1 a 8 y en la composición del documento $\text{\LaTeX}$ (plantilla, tabla `booktabs`, figura, diagrama TikZ de la Sección 4), sobre el análisis y las decisiones de contenido que son de mi autoría.
- **Búsqueda de referencias propias:** búsqueda y verificación de DOI de las dos referencias declaradas como propias en la Sección de Referencias ([6], [7]), distintas de la bibliografía obligatoria de PE-U4.

El análisis de los resultados, la interpretación de las tendencias, la propuesta de integración al PFC y las conclusiones son responsabilidad exclusiva del autor de este informe.

Anexo: formalización matemática

Las siguientes expresiones son las exigidas por la guía de la actividad y se citan en el texto de las secciones anteriores.

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \quad S_{\text{máx}} = \lim_{N \rightarrow \infty} S(N) = \frac{1}{1-p} \quad (1)$$

$$e = \frac{\frac{1}{S} - \frac{1}{N}}{1 - \frac{1}{N}}, \quad N > 1 \quad (2)$$

$$E(N) = \frac{S(N)}{N} \quad (3)$$

$$T_{\text{sec}}(n) = \alpha n, \quad T_{\text{dist}}(n) = \beta + \frac{\gamma n}{N}, \quad n^* = \frac{\beta}{\alpha - \frac{\gamma}{N}}, \quad \text{válido si } \alpha > \frac{\gamma}{N} \quad (4)$$

Referencias

- [1] G. M. Amdahl, “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities,” en *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67 (Spring))*, Atlantic City, NJ, USA: ACM, 1967, págs. 483-485. DOI: [10.1145/1465482.1465560](https://doi.org/10.1145/1465482.1465560).
- [2] M. Zaharia et al., “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, n.º 11, págs. 56-65, 2016. DOI: [10.1145/2934664](https://doi.org/10.1145/2934664).
- [3] Federal Communications Commission, *CGB – Consumer Complaints Data*, <https://opendata.fcc.gov/d/3xyp-aqkj>, Accedido: agosto 2026. Fuente del dataset usado en PE-U4 y reutilizado en este informe (commit declarado en la Sección 2)., 2026.
- [4] A. H. Karp y H. P. Flatt, “Measuring Parallel Processor Performance,” *Communications of the ACM*, vol. 33, n.º 5, págs. 539-543, 1990. DOI: [10.1145/78607.78614](https://doi.org/10.1145/78607.78614).
- [5] M. D. Hill y M. R. Marty, “Amdahl’s Law in the Multicore Era,” *Computer*, vol. 41, n.º 7, págs. 33-38, 2008. DOI: [10.1109/MC.2008.209](https://doi.org/10.1109/MC.2008.209).
- [6] C. Dünner, T. Parnell, K. Atasu, M. Sifalakis y H. Pozidis, “Understanding and Optimizing the Performance of Distributed Machine Learning Applications on Apache Spark,” en *Proceedings of the 2017 IEEE International Conference on Big Data (BigData)*, Referencia aportada por búsqueda propia del estudiante (no consta en la bibliografía obligatoria de PE-U4); sustenta el mecanismo de sobrecarga por serialización Python-JVM discutido en la Sección 4., Boston, MA, USA: IEEE, 2017, págs. 331-338. DOI: [10.1109/BigData.2017.8257942](https://doi.org/10.1109/BigData.2017.8257942).
- [7] Z. Shojaee Rad y M. Ghobaei-Arani, “Data Pipeline Approaches in Serverless Computing: A Taxonomy, Review, and Research Trends,” *Journal of Big Data*, vol. 11, pág. 82, 2024, Referencia aportada por búsqueda propia del estudiante (no consta en la bibliografía obligatoria de PE-U4); sustenta el modelo de dimensionamiento elástico discutido en la Sección 6. DOI: [10.1186/s40537-024-00939-0](https://doi.org/10.1186/s40537-024-00939-0).